Stler Tasks

Technical Documentation

Version: 1.0 (April 2026)

Live URL: stler-tasks.vercel.app

Repository: github.com/JuliaSivridi/Tasks

Stack: React 18 · TypeScript 5 · Vite 7 · Google Sheets API v4

> **Inspiration:** This project was inspired by **Todoist** (todoist.com) — one of the best personal task management apps. Stler Tasks aims to deliver a similar UX as a serverless PWA backed solely by Google Sheets, with no monthly subscription.

Table of Contents

1. Overview
2. Tech Stack
3. Project Architecture
4. Data Model
5. State Management
6. API Layer
7. Sync & Offline
8. Authentication
9. UI & Views
10. PWA & Service Worker
11. Setup & Deployment

1. Overview

Stler Tasks is a personal task manager delivered as a **Progressive Web App (PWA)**. It runs in any modern browser and installs on Android and iOS home screens as a standalone app. There is no dedicated backend server — all persistent data lives in a **Google Spreadsheet** accessed via the Google Sheets API v4.

Inspired by **Todoist**, the app provides a familiar task management UX: priority levels, folders, labels, recurring tasks, subtasks, and an upcoming calendar view — all without any subscription fee or third-party cloud service beyond Google.

Key Design Goals

- **Zero backend** — Google Sheets as a database; no Node/Express/serverless functions
- **Offline-first** — full read/write via IndexedDB (Dexie.js); sync queue flushes when online
- **Mobile-first** — 16 px minimum font, touch-friendly controls, installable PWA
- **Dark mode** — follows OS preference; high-contrast elements throughout

2. Tech Stack

Layer	Technology	Version	Notes
UI framework	React	18	Concurrent mode; hooks only
Language	TypeScript	5	Strict mode
Build tool	Vite	7	ES modules, HMR
Styling	Tailwind CSS	3	Dark mode via `media` strategy
Component library	shadcn/ui	latest	Radix primitives + Tailwind
State	Zustand	5	Multiple slices; no context boilerplate
Database	Google Sheets API v4	—	Row-per-entity; REST
Auth	Google Identity Services	—	OAuth 2.0 implicit grant; GIS token
Offline storage	Dexie.js	4	IndexedDB wrapper
Drag & drop	@dnd-kit	6	Sortable + pointer/keyboard sensors
Date utilities	date-fns	3	Parsing, formatting, week calc
PWA	vite-plugin-pwa	—	Workbox GenerateSW strategy
Hosting	Vercel	—	Auto-deploy from `main` branch

3. Project Architecture

```
src/
├── api/ # Google Sheets REST wrappers
│ ├── sheetsClient.ts # auth header, sheetsRequest(), findRowIndex()
│ ├── tasksApi.ts # CRUD for tasks sheet (A:Q — 17 columns)
│ ├── foldersApi.ts # CRUD for folders sheet
│ ├── labelsApi.ts # CRUD for labels sheet
│ ├── settingsApi.ts # read/write JSON blob in settings!A1
│ └── spreadsheetSetup.ts # create spreadsheet + sheets on first run
├── services/
│ ├── db.ts # Dexie schema (tasks, folders, labels, syncQueue)
│ ├── syncService.ts # initialLoad, scheduleFlush, fullSync
│ ├── recurrenceService.ts # getNextDueDate() for recurring tasks
│ └── authService.ts # GIS token request / silent refresh
├── store/ # Zustand stores
│ ├── tasksStore.ts # tasks[], addTask, updateTask, deleteTask, setTaskExpanded
│ ├── foldersStore.ts # folders[], CRUD
│ ├── labelsStore.ts # labels[], CRUD + updateLabel
│ ├── uiStore.ts # selectedView, selectedFolderId/LabelId/Priority, sidebar
│ ├── syncStore.ts # isSyncing, lastSyncAt
│ ├── authStore.ts # user, accessToken, spreadsheetId
│ └── prefsStore.ts # sectionOpen (sidebar collapse), loads/saves to settings sheet
├── hooks/
│ ├── useTasks.ts # useUpcomingGroups, useFilteredRootTasks, useAllTasks,
│ │ # useLabelTasks, usePriorityTasks, useCompletedTasks
│ └── useSync.ts # online/offline event listeners, auto-flush
├── components/
│ ├── layout/ # AppShell, Header, Sidebar
│ ├── tasks/ # TaskItem, TaskList, TaskChildren, TaskCreateModal,
│ │ # TimePickerDialog
│ ├── common/ # ConfirmDialog
│ └── ui/ # shadcn/ui primitives (button, checkbox, dialog …)
├── types/ # Task, Folder, Label TypeScript interfaces
└── utils/ # constants (TASK_COL, TASK_RANGE), sheetsMapper, dateUtils
```

Data Flow

1. **App start** → `ensureSpreadsheet()` → `initialLoad()` → `prefsStore.load()`
2. `initialLoad()` reads all rows from Sheets → populates Dexie tables → hydrates Zustand stores
3. User mutations call store actions → write to Dexie → enqueue a sync operation → call `scheduleFlush()`
4. `scheduleFlush()` debounces 800 ms then drains the sync queue to Google Sheets

5. On reconnect (`online` event) → `fullSync()` re-reads Sheets and merges with local state

4. Data Model

All data lives in the user's Google Spreadsheet named **db_tasks** (found or auto-created on first login). It contains four sheets:

tasks (columns A–Q)

Col	Field	Type	Notes
A	id	string	`tsk_xxxxxxxx` (8 random hex chars)
B	parent_id	string	Empty for root tasks
C	folder_id	string	Defaults to `fld-inbox`
D	title	string	
E	status	`pending | completed`	
F	priority	`urgent | important | normal`	
G	deadline_date	string	`YYYY-MM-DD`
H	deadline_time	string	`HH:MM`, optional
I	is_recurring	boolean	
J	recur_type	`days | weeks | months`	
K	recur_value	number	Interval (e.g. 2 = every 2 weeks)
L	labels	string	Comma-separated label IDs
M	sort_order	number	Multiplied by 10; integer steps
N	created_at	ISO 8601	
O	updated_at	ISO 8601	Used for conflict resolution
P	completed_at	ISO 8601	
Q	is_expanded	`TRUE | FALSE`	Subtask tree expand state, synced across devices

folders

Field	Type	Notes

id	string	`fld-inbox` is always present
name	string	
color	string	Hex color
sort_order	number	
created_at / updated_at	ISO 8601	

labels

Field	Type	Notes
id	string	
name	string	
color	string	Hex color
created_at / updated_at	ISO 8601	

settings

Single cell `A1` stores a JSON blob with user preferences:

```
{
  "sectionOpen": {
    "priorities": true,
    "folders": true,
    "labels": false
  }
}
```

5. State Management

All state lives in Zustand stores. There is no React Context for data (only for UI primitives from shadcn/ui).

Store	Responsibility
`tasksStore`	tasks array; `addTask`, `updateTask`, `deleteTask`, `completeTask`, `setTaskExpanded`, `getChildren`, `moveTasksToFolder`, `stripLabelFromTasks`
`foldersStore`	folders array; full CRUD
`labelsStore`	labels array; full CRUD including `updateLabel(id, Partial<Label>)`
`uiStore`	`selectedView`, `selectedFolderId`, `selectedLabelId`, `selectedPriority`, `sidebarOpen`, `createTaskOpen`
`syncStore`	`isSyncing`, `lastSyncAt`
`authStore`	`user`, `accessToken`, `tokenExpiry`, `spreadsheetId` (persisted to localStorage)
`prefsStore`	`sectionOpen` per sidebar section; `load()` from Sheets, `toggleSection()` with 1 s debounced save

Sync Trigger Pattern

All store mutations follow this pattern to avoid circular dependencies between the stores and `syncService`:

```
// Inside any store mutation:
void import('@/services/syncService').then(({ scheduleFlush }) => {
  scheduleFlush()
})
```

Since both modules are already loaded at call time, the dynamic import resolves immediately from the module cache.

View Types

```
type SelectedView =
  | 'upcoming'   // day-grouped tasks with deadlines + week strip
  | 'all'        // all pending tasks, sorted by priority → deadline
  | 'folder'     // tasks in selectedFolderId, drag-and-drop reorder
  | 'label'      // tasks with selectedLabelId
```

```
    | 'priority'   // tasks with selectedPriority (urgent/important/normal)
    | 'completed'  // completed tasks, sorted by completion time desc
```

6. API Layer

sheetsClient.ts

Central HTTP wrapper for all Sheets API calls:

- `sheetsRequest(method, path, body)` — attaches Bearer token, calls `https://sheets.googleapis.com/v4/spreadsheets/{id}/{path}`
- `findRowIndex(sheet, entityId)` — scans column A, caches row numbers in a `Map` to avoid repeat scans
- Token refresh: if `tokenExpiry - now < 60 s`, calls `authStore.refreshToken()` before the request

Entity APIs

File	Functions
tasksApi.ts	`fetchAllTasks`, `appendTask`, `updateTask`, `ensureHeader`
foldersApi.ts	`fetchAllFolders`, `appendFolder`, `updateFolder`, `deleteFolder`
labelsApi.ts	`fetchAllLabels`, `appendLabel`, `updateLabel`, `deleteLabel`
settingsApi.ts	`loadSettings`, `saveSettings` (auto-creates settings sheet on first write)

Row Mapping

`src/utils/sheetsMapper.ts` converts between Sheets rows (string arrays) and TypeScript objects:

- `rowToTask(row)` — index constants from `TASK_COL` (e.g. `TASK_COL.IS_EXPANDED = 16`)
- `taskToRow(task)` — produces a 17-element array (`TASK_RANGE = 'tasks!A:Q'`)

7. Sync & Offline

Dexie Schema (db.ts)

```
db.version(1).stores({
  tasks:     '&id, parent_id, folder_id, status, priority, updated_at',
  folders:   '&id',
  labels:    '&id',
  syncQueue: '++id, entityType, operation, entityId',
})
```

Sync Queue

Every mutation enqueues an operation:

```
enqueue(entityType: 'task' | 'folder' | 'label',
        operation:  'create' | 'update' | 'delete',
        entityId:   string,
        payload:    Record<string, unknown>)
```

The queue is drained FIFO during flush. Completed operations are deleted from Dexie.

scheduleFlush

Debounced 800 ms. Called after every mutation. On flush it iterates the sync queue and calls the appropriate Sheets API function for each entry.

Conflict Resolution

On `fullSync()`, local and remote records are compared by `updated_at`. The newer version wins. `setTaskExpanded` deliberately *does not* bump `updated_at` so that expand/collapse events never overwrite real edits.

Offline Indicator

`useSync` hook listens to `window.addEventListener('online' | 'offline')`. Going online triggers `fullSync()` automatically.

8. Authentication

Authentication uses **Google Identity Services (GIS)** with the OAuth 2.0 implicit token grant (no server-side secrets).

- On app load, GIS attempts a silent token request (`prompt: ''`). If it fails, the login screen appears.
- Access token stored in `authStore` and persisted to localStorage via Zustand `partialize`. Expires in ~1 hour.
- `spreadsheetId` is stored in localStorage so users don't lose their data reference after a page refresh.
- Token refresh: triggered automatically inside `sheetsClient` when expiry is < 60 s away.

> **Vercel setup:** Add `https://your-app.vercel.app` to *Authorized JavaScript Origins* (not Redirect URIs) in Google Cloud Console. Add your Google account as a test user in the OAuth consent screen.

9. UI & Views

Theme

- Dark mode follows OS preference (`darkMode: 'media'` in Tailwind config)
- Dark background: `#1c1c1c`
- Primary color: `#e07e38` — `hsl(25, 75%, 55%)` in light mode
- Minimum font size: 1 rem everywhere (xs and sm Tailwind tokens both mapped to 1 rem)

Upcoming View

- Tasks with deadlines grouped by day: Overdue / Today / Tomorrow / This week / Future
- **Week navigation strip** — 7-day horizontal bar; `<` / Today / `>` scroll the task list and update the strip; scrolling the list auto-advances the strip via IntersectionObserver
- Priority + label filter bar above the task list

Task Item (two-row design)

- **Row 1:** expand/collapse chevron · checkbox (priority-colored) · title · action icons
- **Row 2:** recurring icon · deadline label (color-coded) · label dots · folder name · subtask count
- Action icons always visible; mobile uses Clock + `···` overflow menu with submenus

Sidebar

Three collapsible sections — **Priorities**, **Folders**, **Labels** — with chevron toggles. Collapse state is synced to Google Sheets (`settings!A1`) and persists across devices.

Drag & Drop

@dnd-kit powers both root-task reorder (FolderView) and subtask reorder (TaskChildren). Dragging a task more than 50 px to the right reparents it under the task it's dragged over.

Recurring Tasks

Completing a recurring task does *not* change its status to `completed`. Instead, `recurrenceService.getNextDueDate()` advances the `deadline_date` by the configured interval. A "Postpone" button in the TimePickerDialog is also available for recurring tasks.

10. PWA & Service Worker

Configured via `vite-plugin-pwa` with Workbox's `GenerateSW` strategy.

- All static assets (JS, CSS, fonts) are precached on install
- Network-first strategy for API calls; cache-first for static assets
- Web app manifest includes name, short name, icons (192×192, 512×512), display `standalone`
- Android: Chrome auto-prompts for installation; iOS: Safari → Share → Add to Home Screen

11. Setup & Deployment

Prerequisites

- Google account
- Google Cloud project with **Google Sheets API v4** and **Google Drive API v3** enabled
- OAuth 2.0 Client ID (type: Web application)
- Node.js ≥ 18

Google Cloud Console

1. Enable **Google Sheets API v4**
2. Create an **OAuth 2.0 Client ID** → Web application
3. Add to **Authorized JavaScript Origins:**

   ```
   http://localhost:5173
   https://your-app.vercel.app
   ```

4. Add your account as a **test user** in OAuth consent screen

Local Development

```
git clone https://github.com/JuliaSivridi/Tasks.git
cd Tasks
npm install
```

Create `.env` in the project root:

```
VITE_GOOGLE_CLIENT_ID=your-client-id.apps.googleusercontent.com
```

```
npm run dev    # http://localhost:5173
npm run build  # production build → dist/
```

Vercel Deployment

1. Import the repository at **vercel.com**
2. Add environment variable in Settings → Environment Variables:
   `VITE_GOOGLE_CLIENT_ID`
3. Every push to `main` triggers automatic deployment

First Run

On first launch, `ensureSpreadsheet()` creates a new Google Spreadsheet with four sheets (`tasks`, `folders`, `labels`, `settings`) and a default Inbox folder. The spreadsheet ID is saved to localStorage for subsequent sessions.